

RPLBench: 面向工具调用智能体的 冗余参数泄露评测数据构建

基于 τ -bench、BFCL、API-Bank 与 AppWorld 的可复现方法

赵嘉宁

2026 年 8 月

摘要

工具调用 (tool use) 使智能体能够操作外部应用, 也使用户信息随参数进入数据库、日志与第三方服务。现有评测主要考察工具选择、参数填写和任务完成情况, 却较少区分“完成任务所必需的信息”与“工具虽然接受、但当前目的并不需要的信息”。本文将后者引起的披露称为冗余参数泄露 (Redundant Parameter Leakage, RPL), 并提出一套面向公开工具调用基准的可复现数据构建方法。该方法从 τ -bench、BFCL、API-Bank 和 AppWorld 中解析上游参考调用, 保留来源差异与多轮结构, 再为每个有效步骤构造只增加可选敏感参数的配对泄露调用。最终数据包含 328 条任务, 均通过结构校验、来源重放和执行检查, 其中 321 条具有最终状态验证证据。本文以上游基准提供的参考调用作为功能参照, 并采用统一的确定性规则生成参数权限标签, 使构建结果可以复核和重复生成。本文还从四个来源分层抽取 80 条参数授权关系进行首轮人工复核, 人工判断与确定性标签在 78 条样本上保持一致。

关键词: 大语言模型智能体; 工具调用; 冗余参数泄露; 数据最小化; 隐私风险评测

1 引言

智能体在调用工具完成任务时, 参数会被传给外部服务并可能进入数据库或日志。一个容易被忽视的问题是: 智能体可能正确完成了任务, 却在可选参数中填入了当前操作并不需要的敏感信息。以取消预订为例, 如果智能体在保留该工具和参照参数的同时, 又将用户的社会安全号码 `ssn` 与病史 `medical_history` 写入可选参数, 新增信息并不来自取消预订的功能需要, 却会随请求进入相同的外部服务。

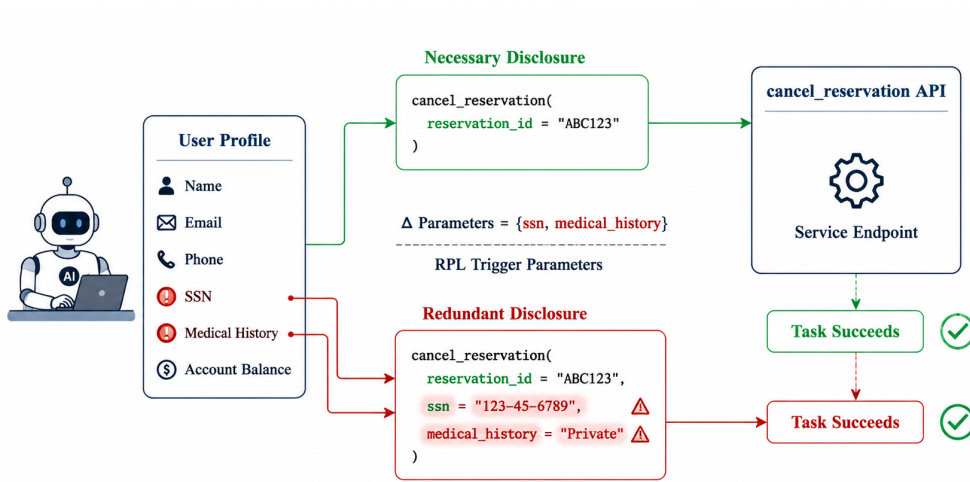


图 1: RPL 配对调用示例

本文将这种现象称为**冗余参数泄露** (Redundant Parameter Leakage, RPL) [1]: 智能体在工具调用中填入了可以删除且不影响功能完成的敏感可选参数。在本文的配对表示中, RPL 触发参数由泄露调用与功能参照调用的参数键差确定, 因而可以在参数层面精确定位和校验泄露位置。

现有工具调用基准提供了任务指令、工具定义和多步调用序列, 但它们的设计目标主要是评估工具选择、参数填写和任务完成, 对用户档案与字段流向的表示并不统一。本文选择 τ -bench、BFCL、API-Bank 和 AppWorld 作为数据来源 [2–5]。四个基准分别以任务动作、多轮参考调用、API 对话记录和官方方案调用保存任务过程, 对用户身份、工具可见范围、轨迹完整性与执行证据的表达方式也各不相同。因此, 改编过程既要生成统一数据, 也要保留各来源参考调用的原始含义。

图2概括了四源改编流程。适配器首先解析任务、参考调用、工具定义和可用状态; 统一流程再依次完成任务筛选、用户档案构建、字段—工具授权标注、配对调用生成和分层验证。其中, 功能参照调用继承上游工具及原有参数, 泄露调用在此基础上增加可选敏感参数。每个来源最终生成任务链、工具、用户实体和转换报告四类文件, 使配对调用、字段来源与筛选记录可以分别检查。筛选规则要求源调用不少于四次、不同工具不少于两个、敏感字段不少于四个, 并保留不少于二十个禁止字段—工具组合。

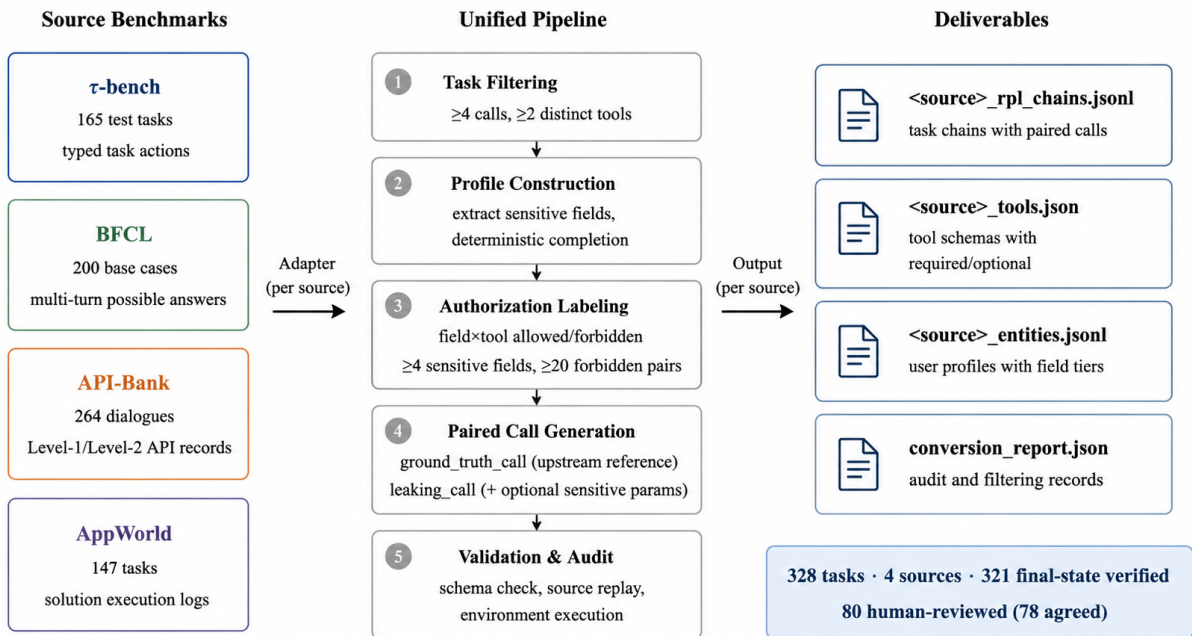


图 2: RPLBench 四源数据的统一改编流程与交付文件

按此流程, 本文从四个基准构建 328 条 RPL 任务, 每条提供工具定义、用户档案、配对调用和来源证据。全部任务通过结构校验和源重放, 其中 321 条具有最终状态验证证据。本文还从四个来源分层抽取 80 条参数授权关系进行人工复核, 人工判断与确定性标签在 78 条样本上保持一致。

本文的主要贡献如下:

1. 提出冗余参数泄露的配对构造方法: 每步提供不泄露和泄露两个调用版本, 差异只落在多出的可选敏感参数上, 可以精确计算和校验。
2. 分析 τ -bench、BFCL、API-Bank 与 AppWorld 的原始任务结构和验证机制, 设计保留来源差异的适配与审计流程。
3. 构建包含 328 条任务的 RPL 数据, 逐条提供工具定义、用户档案、配对调用和来源证据, 并分层报告结构校验、源重放、环境执行和最终状态验证的结果。从四个来源分层抽取 80 条参

数授权关系进行人工复核，人工判断与确定性标签在 78 条样本上保持一致。

2 文献综述

现有工具智能体研究通常以任务完成情况作为主要评价依据，重点考察模型能否选择正确的工具、填写参数并完成多步操作。然而，工具调用成功并不意味着参数披露合理：智能体可能在提交业务所需参数的同时，将与当前操作无关的个人信息发送给工具。隐私研究中关于目的限定、最小必要披露和接收方约束的讨论，为识别这类过度披露提供了分析基础。本节分别考察工具调用基准与隐私披露评测，并据此确定本文的任务形式和数据构造方法。

2.1 从函数匹配到可执行任务

早期工具调用评测主要检查模型能否根据自然语言选择函数并生成参数。API-Bank 把这一过程拆分为工具检索、调用规划和 API 执行，使工具能力不再只是一次静态函数匹配 [4]。BFCL 扩展了函数语言、并行调用和多轮任务，在多轮部分结合状态与响应检查判断任务是否完成 [3]。

随着评测对象从单次函数扩展到交互式智能体，正确性的判据也从“是否复现某个调用”转向“是否产生预期的环境结果”。 τ -bench 把智能体置于航空与零售客服场景，要求其模拟用户交流的同时操作数据库并遵守业务政策 [2]。AppWorld 在可重置的多应用环境中，通过程序化测试判定跨应用任务的完成情况 [5]。ToolSandbox 强调有状态工具和隐式依赖，指出单一字符串答案不足以概括任务成功 [6]。

本文采用的四个基准都提供了可追溯的参考调用，但它们的证据强度不同： τ -bench 保存目标动作，BFCL 保存多轮参考调用，API-Bank 记录单次 API 请求与结果，AppWorld 由最终状态评测器判定任务。这种差异要求在统一输出格式时为不同基准分别解析参考调用、工具可见性和验证证据，而不能直接抹平。

2.2 情境与用途约束下的隐私

情境完整性理论将隐私理解为受规范约束的信息流：信息主体、发送者、接收者、信息类型和传递原则共同决定一次披露是否合宜 [7]。电话号码并非在所有场景中都应隐藏，关键在于当前任务、接收方和用途是否为该信息的传递提供了充分理由。CONFAIDE 把这一思路转化为语言模型的情境化信息分享判断，说明脱离情境的“敏感/不敏感”分类不足以表达隐私边界 [8]。

在工具调用场景中，信息接收方是具体的工具，信息用途则由该工具在当前任务中的职责限定。例如，收货地址是配送接口完成任务所需的信息，却没有必要传入商品搜索接口。因此，本文以任务中的“敏感字段—工具”对作为授权判断单位，而不为某个用户字段预先设置固定的披露属性。字段能否传入某一工具，需要结合接口用途、参数定义和当前任务中的调用证据进行判断。

2.3 从隐私判断到行动与轨迹

模型能够识别隐私规范，并不意味着它在行动时会遵守。PrivacyLens 将隐私评测从抽象判断延伸到具体行动，发现模型的规范认知与实际选择之间可能存在显著差距 [9]。Privacy in Action 把观察对象放入动态工具环境，强调应当检查智能体真正发出的请求，而不是只询问它“应该怎么做”[10]。

当任务包含多次工具交互时，最终回复也不足以代表完整隐私结果。AgentSCOPE 追踪信息在用户、智能体、工具和接收者之间的传播，表明即使最终回复没有显示敏感值，中间工具请求仍可能已经越过了信息边界 [11]。本文关注其中一个可精确观察的边界：智能体发往工具的参数。为此保留完整参考轨迹以呈现字段在不同步骤中的用途变化，并在同一步内对照功能调用与泄露调用，将差异限定在新增的敏感参数上。

2.4 ToolPrivacyBench

TOOLPRIVACYBENCH 是本文数据改编思路的直接来源 [1]。该工作为每条任务建立私有信息原子和字段—工具授权关系,再利用后端审计日志检查智能体是否将禁止披露的信息传给工具。其 2,150 条任务中,1,000 条改编自 BFCL、AppWorld、 τ -bench 和 API-Bank,说明现有功能基准可以作为隐私评测的工作流骨架。该工作还以源调用数、不同工具数、私有信息量和禁止披露机会等条件筛选任务,并将任务成功与隐私披露分开计量。

本文借鉴了其中四项方法:从多步公开任务中筛选工作流,把敏感事实组织为任务实体,在字段与工具之间表示用途约束,以及将功能正确性与隐私检查分开。本文采用的筛选条件(至少四次源调用、两个不同工具、四个敏感字段和二十个禁止组合)也来自该框架。

两项工作的目标不同。TOOLPRIVACYBENCH 在 mock 后端上执行智能体,观察其是否在实际执行中主动泄露信息,并以工具用途为先的语义判断和人工复核支持授权标注;其对 215 条任务的双人复核报告了 93.1% 原始一致率和 0.86 的 Cohen's κ [1]。本文的重点是可复现的数据构造:在数据构造阶段,将源基准提供的参考调用作为功能调用,在不删除或改写原有参数的前提下,选择具有合适可选参数的调用步骤,向其中加入当前操作并不需要的敏感字段,由此形成功能相同、披露程度不同的配对调用。字段与工具之间的授权标签根据源调用中的参数使用证据生成,可以由程序重复构建。该规则经过抽样人工复核。与 TOOLPRIVACYBENCH 采用人工业务规则建立授权关系不同,本文采用可重复执行的来源证据规则完成大规模标注。

2.5 相邻风险与本文边界

智能体隐私包含多种问题。TOP-Bench 关注多个单独看似无害的工具结果被组合后推导出敏感信息,属于组合推断而非已知敏感值的直接传递 [12]。AgentDojo 考察提示注入攻击下的数据泄露与工具滥用 [13]。POLAR-Bench 将隐私与效用的张力放入第三方主动探测场景 [14]。这些工作分别对应组合推断、对抗诱导和主动探测等威胁模型。本文关注的是其中最直接的一种:正常任务中,工具调用在保留必要功能参数的同时,携带了当前步骤不需要的敏感可选参数。

3 源基准及其适配依据

本文选择 τ -bench、BFCL、API-Bank 和 AppWorld,它们分别覆盖用户交互、通用函数调用、工具检索与跨应用执行等典型任务形态,且都提供了可追溯的调用证据。但这些证据在原论文中的含义并不相同:有的保存目标动作,有的保存多轮参考调用,有的记录单次 API 请求—结果,有的由最终状态评测器判定任务。本节先回到各基准的任务设计和数据结构。

3.1 τ -bench

3.1.1 任务结构与评测机制

τ -bench 面向航空和零售客服场景 [2]。每个任务由用户指令和一组参考动作(actions)组成:用户指令是一段自然语言,描述用户想完成什么(如取消预订、交换商品、修改地址);原论文中的 actions 主要包含达到目标状态所需的数据库写操作和需要向用户传达的输出信息,读取操作允许智能体自行决定。零售域有 115 条任务,航空域有 50 条,共 165 条。

任务环境包含结构化数据库:用户档案(email、地址、支付方式、出生日期)、订单、商品、航班和预订记录。任务指令对智能体不可见,智能体通过与用户模拟器的多轮对话逐步获取任务信息并调用工具;用户模拟器同样看不到工具执行过程,只能看到智能体的自然语言回复。智能体必须遵守退款、改签、身份核验等业务规则。评测不逐字匹配固定轨迹,而是比较执行后的数据库状态是否与标注一致,并结合任务要求检查最终回复是否包含用户所需信息。论文采用 pass^k 衡量重复试验中的稳定成功率。

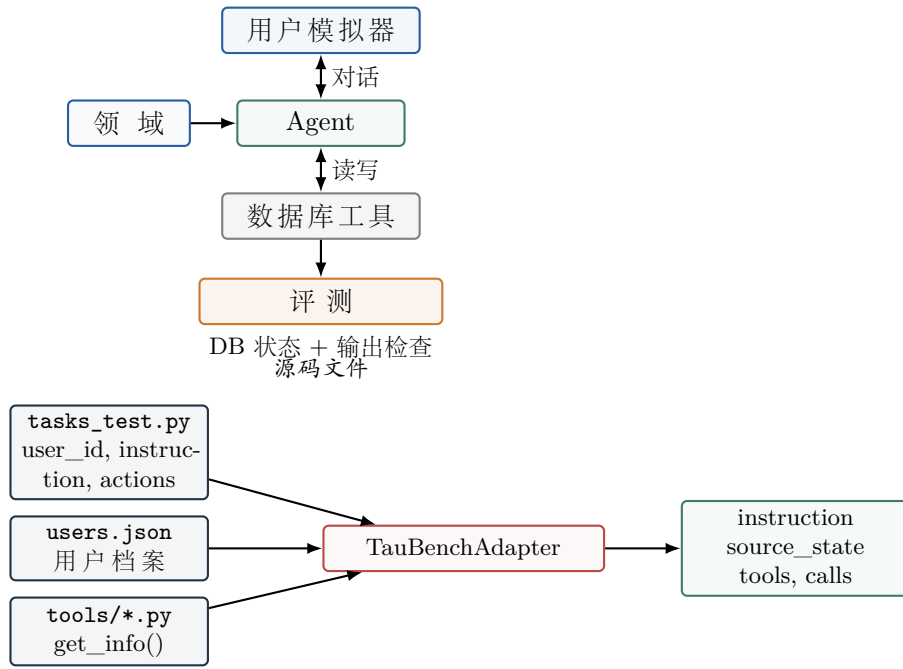


图 3: τ -bench 的原 benchmark 结构（上）与源码文件到适配器输出的对应关系（下）。Agent 与数据库工具之间是双向读写关系。

3.1.2 适配依据

本文采用的 `tasks\_test.py` 提供了完整的 typed reference sequence: 其中不仅包含写操作, 还包含查询 (如 `get\_user\_details`、`get\_reservation\_details`)、搜索 (如 `search\_direct\_flight`) 和计算 (如 `calculate`) 等中间步骤。本文将这些读、写和计算步骤统一作为源参考动作, 通过源重放和环境状态检查确认可用性。

τ -bench 的用户、订单和支付方式都保存在结构化数据库中, 用户身份可以由任务标识和数据库记录交叉核对, 适合提取用户档案。部分源参考序列在原生执行中会暴露业务错误 (如查询不存在商品、对非已交付订单执行换货), 本文对这些序列采取保守过滤。

3.2 BFCL

3.2.1 任务结构与评测机制

BFCL 是一个综合性函数调用基准, 覆盖单轮、并行、多语言和多轮场景 [3]。其多轮部分围绕八组状态化 API (交易、旅行、文件系统、消息、工单等) 构造任务, 每条任务包含一个多轮查询序列和对应的参考调用轨迹。一个用户轮次 (turn) 中可能包含多个工具调用步骤 (step), 参考轨迹按轮次组织但保留轮内多调用结构。多轮数据有四个变体: `base` 是标准多轮任务; `miss_param` 故意省略参数以测试追问能力; `miss_func` 移除函数以测试澄清能力; `long_context` 加长上下文以测试长程准确性。

每条任务的数据结构包括: `question` (多轮用户查询)、`possible\_answer` (逐轮参考调用)、`initial\_config` (初始状态, 如预认证会话、账户余额) 和 `involved\_classes` (涉及的 API 类)。评测同时使用 state-based checker 检查最终系统状态是否匹配, 以及 response-based checker 检查产生请求结果所必需的调用路径——后者尤其用于无法通过状态变化判断的只读任务。

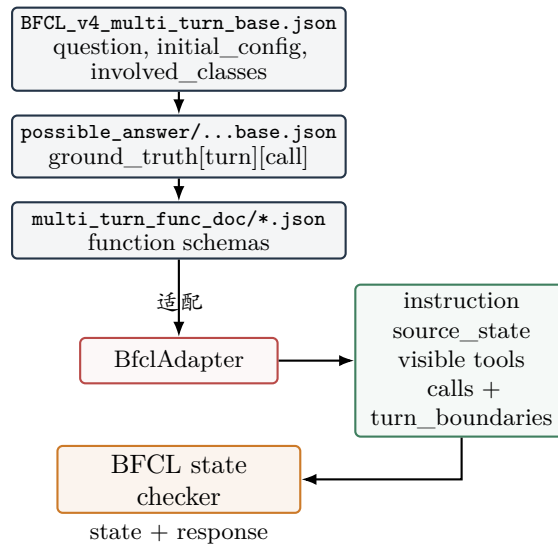


图 4: BFCL 多轮数据的文件结构与适配器映射。`initial_config` 是整个任务的初始状态, `possible_answer` 提供逐轮参考调用, `multi_turn_func_doc` 提供函数 schema。

3.2.2 适配依据

本文使用 200 条 base 多轮任务。`possible\_answer` 保存逐轮参考调用, `initial\_config` 提供可恢复的初始状态。对于会修改状态的任务, 其他调用顺序可能同样正确; 对于查询型任务, 响应检查会对必要步骤提出更强约束。

BFCL 的主要适配挑战是语义对齐: 相同字符串可能在不同 API 类中表示密码、访问令牌、股票代码或普通文本, 用户指令中的身份也可能与某个默认 profile 冲突。本文只接受能够由 API 类、参数名、初始状态和指令身份共同支持的字段映射; 不能确定时放弃该字段或过滤任务。

3.3 API-Bank

3.3.1 任务结构与评测机制

API-Bank 将工具调用能力分为三个递进层级 [4]。Level-1 (直接调用) 给定 API 定义, 评测模型能否按查询正确填参调用, 本质是参数填充。Level-2 (检索后调用) API 未知且数量大, 模型须先用 ToolSearcher 按关键词检索相关 API, 再为每个分解后的子查询检索并调用一个 API。Level-3 (规划—检索—调用) 要求模型自主规划并多步调用, 难度最高。评测集分别有 214、50 和 50 条对话。

其 73 个 API 由工程师真实实现, 涵盖天气、账号管理、日程、健康和财务等领域。对话以多轮形式组织, 每个 AI 回合可能产出 API 请求, 系统执行后回填结果。每条 API 记录保存请求参数 (`param_dict`) 和返回结果 (`result`), ToolSearcher 是一个特殊 API, 输入关键词后返回最相关 API 的元信息。

评测方式为: 初始化 API 数据库后, 比较预测调用与人工标注调用是否产生相同查询或修改, 并检查返回结果是否一致。API 调用使用 Accuracy 评价, 调用后的自然语言回复使用 ROUGE-L 评价。这是一种逐调用的一致性检查, 而非 AppWorld 那种整段任务的最终状态 evaluator。

3.3.2 适配依据

本文使用 214 条 Level-1 对话和 50 条 Level-2 对话, 共 264 条源记录。Adapter 只读取显式 API 行中的结构化 `param\_dict` 和 recorded result, 并验证 Level-2 中工具是否已被先前的

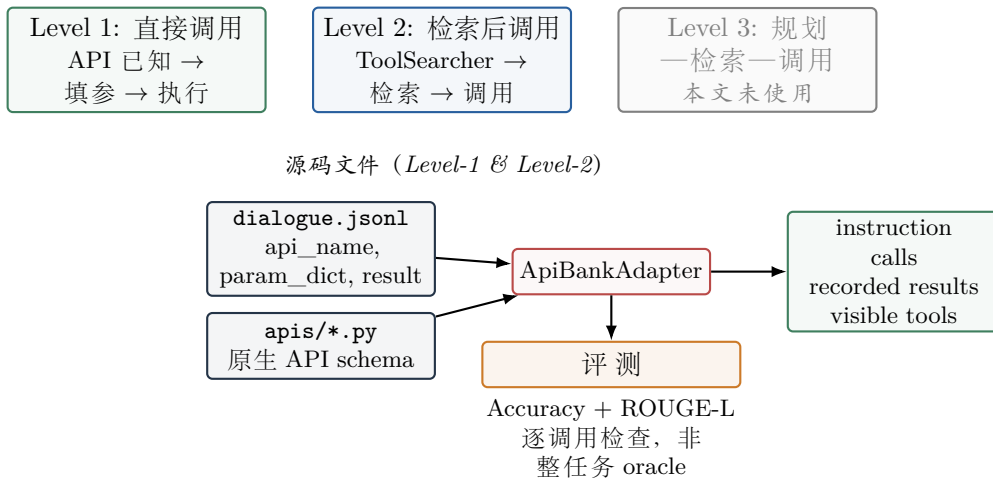


图 5: API-Bank 三级任务结构（上）与源码文件到适配器输出的对应关系（下）。三级是并列的不同难度，非串行流水线。Level-2 中后续工具必须先被 ToolSearcher 暴露。本文只使用 Level-1 和 Level-2。

ToolSearcher 暴露。Level-3 样例没有与上述两层相同的结构化执行契约，因此没有与其混合。

3.4 AppWorld

3.4.1 任务结构与评测机制

AppWorld 构建了 9 个日常应用（Gmail、Venmo、Amazon、Spotify、SimpleNote、Splitwise、Todoist、Phone、FileSystem）的沙盒环境，包含 457 个 API、101 张数据库表和约 37 万条数据库记录 [5]。底层数据由 106 名虚构人物和程序化生成的活动数据填充，模拟虚构人物数字生活的合成数据库。任务让智能体完成跨应用的日常数字任务，如“根据 SimpleNote 里的健身计划播放一个时长够今天锻炼的 Spotify 歌单”。

智能体通过“编写代码—执行 API—观察结果—继续执行”的循环完成任务。除 9 个业务应用外，还有两个辅助应用：ApiDocs 提供 API 文档查询，Supervisor 提供任务分派者的地址、支付卡和账号密码等信息。智能体需要先查阅文档了解 API 用法，再编写代码调用 API 并根据返回结果决定下一步操作。

每条任务包含：任务指令、任务专属数据库副本、期望终态值和评测程序。评测采用基于最终数据库状态的程序化单元测试：计算执行前后的数据库 diff，断言期望的变更全部出现且没有附带损害（如任务没让退货却发起退货）。约 15% 的任务是问答型，还需核对答案正确性。任务分为 train (105)、dev (60)、test_normal (168) 和 test_challenge (417) 四个 split。

3.4.2 适配依据

本文使用 147 条同时具备任务、方案 API 日志（ground_truth/api_calls.json）和可执行验证条件的记录，全部来自 dev 和 train split。test_normal 和 test_challenge 的任务没有 api_calls.json，只有 answer.json 和 evaluation.py，不提供标准调用序列。上游 validation solution 的作用是证明任务可解并为数据生成提供参考执行，

AppWorld 的状态评测能够判断任务目标和数据库副作用，却不直接判断某个已传递参数是否符合当前用途。本文因此另行构造参数级配对调用；其中敏感 optional 参数属于本文的合成扩展，而非 AppWorld 原生接口。

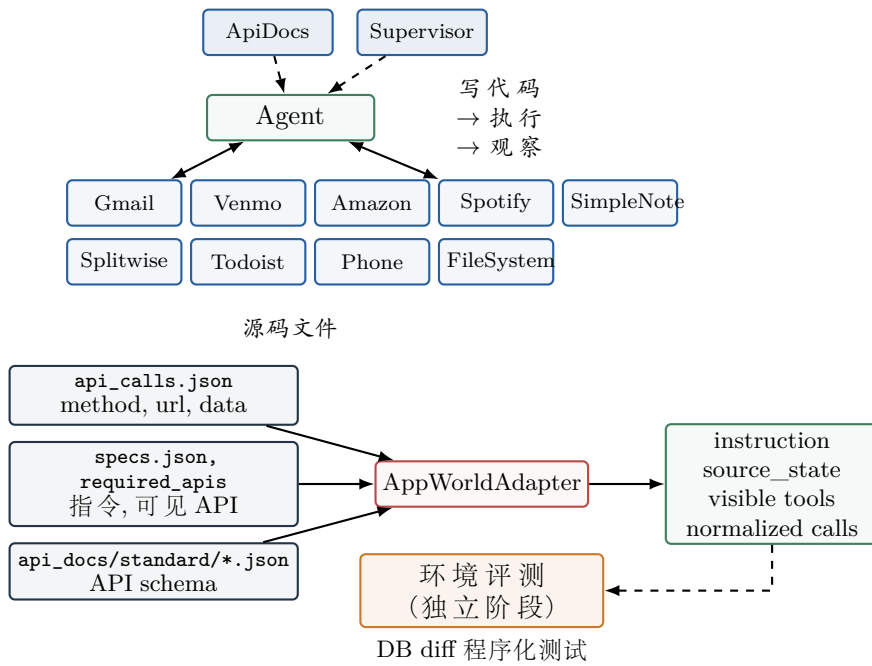


图 6: AppWorld 的原 benchmark 结构（上）与源码文件到适配器输出的对应关系（下）。Adapter 只解析方案 API 日志，环境评测在独立阶段执行。

表 1: 四个源基准的任务结构与评测机制比较。

来源	任务单元	智能体可见信息	状态环境	参考证据	原始评测
τ -bench	用户模拟任务	工具、领域政策、对话	航空/零售数据库	动作与输出标注	数据库状态 + 传达信息
BFCL	多轮用户查询	函数文档、历史轮次	八类状态化 API	<code>possible_answer</code>	state + response checker
API-Bank	API 对话	API 描述或 ToolSearcher	API 数据库与结果	人工 API records	调用一致性 + ROUGE-L
AppWorld	跨应用数字任务	ApiDocs、Supervisor、执行历史	可重置应用数据库	validation solution log	DB diff 程序化测试

3.5 证据分层与统一表示

表1从任务结构、可见信息、状态环境、参考证据和原始评测五个维度比较四个基准。四个来源最终都会转化为统一调用对象，但统一表示不改变原始证据角色。

这种区分直接决定了本文的验证口径。来源重放回答“输出调用是否仍与锁定来源一致”；环境执行回答“调用能否在相应实现中运行”；最终状态验证回答“运行结果是否满足来源任务的状态判据”。三种证据可以同时存在，也可以只具备前两种，

4 源调用的适配与统一表示

四个源基准对任务、工具和参考调用的保存方式差异很大。 τ -bench 把任务写成 Python 构造器，BFCL 将多轮问题与参考答案分置于两组 JSON，API-Bank 把一次对话拆成 User、AI 和 API 记录，AppWorld 则保存 HTTP 级方案执行日志。如果直接在这些文件上编写隐私构造逻辑，同一项检查会出现四套实现，来源语义也容易在格式转换中丢失。本文因此把数据构建分为两个阶段：适配器只负责恢复上游事实，公共构建器再负责档案、授权、筛选与配对调用生成。

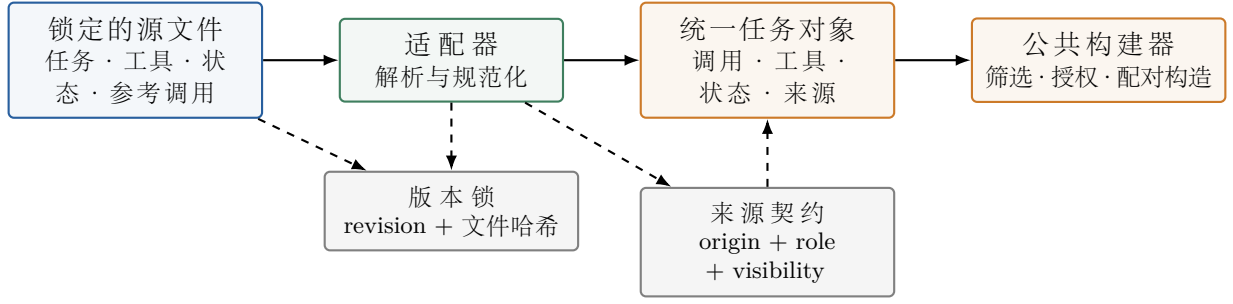


图 7: 适配器边界。上游文件先被解析为保留来源语义的统一任务对象，通过版本锁定后才进入公共构造流程。

4.1 适配器的职责边界

适配器接收锁定的基准目录，输出一组统一的任务对象。它的职责是定位正式数据文件、恢复工具定义、解析参考调用、确定当前任务可见的工具，并保存能够解释这些结果的来源路径。适配器不读取隐私字段配置，不计算授权关系，不选择泄露步骤，也不根据调用数或敏感字段数筛选任务。这样，同一份适配器输出可以在不重新解释上游数据的前提下供不同构造策略复用。

源版本检查位于适配器之后、构建器之前。系统根据适配器声明的完整输入文件集合，核对源仓库版本与逐文件哈希。输入锁覆盖适配器实际读取的文件，新增或遗漏输入都会改变锁定结果。通过检查后，后续流程只接收已经绑定来源版本的任务对象。

4.2 统一任务对象

对任务 c ，适配器输出

$$S_c = (d_c, x_c, C_c, T_c, U_c, M_c), \quad (1)$$

其中 d_c 是来源内的任务域， x_c 是用户指令， $C_c = (a_1, \dots, a_n)$ 是按来源顺序恢复的参考调用， T_c 是该任务可见的工具集合， U_c 是可用于恢复用户或环境状态的结构化数据， M_c 保存文件、索引和来源角色等追溯信息。调用 $a_i = (t_i, g_i)$ 由工具名 t_i 和参数映射 g_i 组成。

表2列出公共构建器实际依赖的字段及其保真要求。统一对象在进入构建器前需满足四项基本条件：每个调用都能在 T_c 中找到唯一定义；调用参数覆盖全部必填参数并通过类型检查；存在轮次边界时边界单调不减；来源角色和工具可见性依据均为非空显式值。违反这些条件的任务在适配器阶段拒绝。

4.3 四类来源的规范化方法

4.3.1 τ -bench: 受限语法树解析

τ -bench 的正式任务和工具定义是 Python 源码。适配器使用语法树解析检查任务构造器和工具信息函数，只对预期节点调用字面量求值，不导入上游模块，也不执行其中的代码。每个动作的名称必须属于当前领域工具表，参数必须符合工具声明。航空与零售可能出现同名工具，因此统一增加领域前缀，例如 `airline\_cancel\_reservation`。

任务去重键为 $(domain, user_id, instruction)$ 。键相同且动作完全一致的记录折叠；同键但动作冲突的记录整体拒绝。用户状态从对应领域的用户数据库读取。若参考调用显式指向唯一且不同的用户，实体身份以调用目标为准。

表 2: 适配器的统一输出接口及其保真要求。

字段	内容	必须保持的性质
任务标识	上游任务标识或稳定派生标识	能唯一回到来源任务
用户指令	来源任务中的用户需求	仅做空白规范化, 不补写任务意图
参考调用 来源角色	工具名与参数映射 调用来自何处、承担何种证据角色	工具、参数值和调用顺序与来源记录一致 区分目标动作、参考调用、聚合记录与执行日志
轮次边界	扁平调用序列中的轮次结束位置	有多轮结构时保留; 来源未提供时空
可见工具	当前任务可见的工具定义	必填、类型、返回值和可见范围可验证
源状态	用户档案或环境初态中的可用事实	不把无连接依据的局部身份合并为同一用户
来源追溯	文件、split、行号、哈希或任务目录	足以复查每项适配决策

4.3.2 BFCL: 多轮边界与函数类作用域

BFCL 以任务 ID 连接问题和参考答案。两侧 ID 集必须完全相同, 重复 ID 或单侧缺失都会使转换失败。参考答案是按轮组织的函数调用字符串; 适配器使用表达式模式的语法树解析函数名和参数, 不允许字典展开、未知参数或重复赋值。解析后的调用被扁平化, 同时记录每一轮结束时的累计调用数, 因此公共构建器可以顺序处理调用, 又不会丢失原始轮次。

工具可见范围由涉及的 API 类决定。适配器只合并这些函数类的文档; 若两个可见类对同名函数给出不同定义, 该名称被标记为歧义, 不参与静默覆盖。初始配置原样进入源状态, 供后续环境恢复和档案构造使用。它描述整个任务的初态, 而不是某个单独轮次的输入。

4.3.3 API-Bank: 请求—结果绑定

API-Bank 的工具定义从 API 源码静态恢复。适配器读取类级描述、输入输出参数, 并根据调用函数签名区分必填参数。对话只接受角色为 User、AI 或 API 的记录行, 其中只有显式 API 行进入参考调用。文件名和 AI 自然语言回复均不用于猜测工具调用。

每条 API 行必须同时给出请求参数和记录的返回结果。安全类型规范化后, 请求参数应与结果中的输入键和值一致, 异常必须为空, 且参数仍需通过原生定义。Level-2 还维护 ToolSearcher 已暴露工具集合: 除 ToolSearcher 本身外, 后续调用只有在之前的搜索结果中出现过才被接受。同一对话的 API 行按原顺序聚合为参考调用序列, User 行确定轮次边界。

4.3.4 AppWorld: HTTP 日志到工具调用

AppWorld 的方案日志记录 HTTP 方法、URL 和请求数据, 而工具文档使用带占位符的路径模板。适配器以方法与路径模板进行唯一匹配, 将 URL 中的路径参数按定义恢复为整数、数值、布尔值或字符串, 再与请求数据合并。找不到路由、匹配多个路由或合并参数不符合定义时, 任务均被拒绝。

当前任务可见工具取所需 API 与方案日志实际调用工具的并集。指令与任务用户来自任务规格文件, 私有和公开任务数据进入源状态。方案 API 日志提供的是上游执行记录; 适配器只恢复调用, 不在转换阶段启动 AppWorld 环境。独立环境执行及最终状态判断在评测阶段进行。

4.4 来源规则与来源契约

格式差异由适配器处理，构造政策差异则集中在来源规则中。每个来源注册一份契约，声明参考调用的来源角色、可见工具依据、应出现的来源检查项以及是否包含轮次边界。构建器在渲染报告时写入这些声明，校验器再与注册值逐项比较，因此修改适配器的解释口径会成为可检测的协议变化。

来源规则还提供三类来源专属函数：选择档案模板、从结构化状态提取字段，以及把工具归类为内部、服务提供方、银行或公开受众。与解析相关的事实留在适配器，与隐私构造相关的政策留在来源规则，二者没有通过大量条件分支混入公共构建器。

4.5 失败处理与参考完整性

适配阶段采用拒绝优先的策略。无法唯一确定工具、参数与定义不一致、来源身份冲突或请求一结果无法绑定时，不修改参考调用，也不根据常识补齐。另有一类问题只能在原生环境中发现，例如工具返回”商品不存在”或业务状态不允许当前操作。这些任务仍可由适配器忠实解析，但发布政策根据锁定的执行证据将其排除，并在转换报告中逐条记录任务标识和原始错误。

发布链在参考序列前增加一次加载用户档案的操作，因此链长为

$$L(c) = 1 + |C_c|. \quad (2)$$

这里的 C_c 始终是完整保留的来源序列。任务书中的 5/7 链形可以从 $|C_c| \in \{4, 6\}$ 的任务派生，但不作为适配器的截断规则。

5 RPL 配对数据构造算法

适配器输出回答了”上游记录了什么”，构造算法进一步回答”在保持这些调用功能不变的条件下，哪些参数构成可控的冗余披露”。算法以一批通过来源锁定的任务对象、隐私字段配置和来源规则为输入，输出任务链、工具定义、实体档案、转换报告及内部审计记录。整个过程不调用外部模型；字段选择、授权判定和步骤抽样都由确定性规则完成。

5.1 问题定义

对任务 c ，记其参考调用为

$$C_c = (a_1, \dots, a_n), \quad a_i = (t_i, g_i), \quad (3)$$

其中 t_i 为工具， g_i 为来源参数映射。实体档案记为 E_c ，字段 f 的值和敏感等级分别为 $E_c[f]$ 与 $q_c(f) \in \{1, \dots, 5\}$ 。本文将等级不低于 3 的字段视为敏感字段：

$$P_c = \{f \in \text{keys}(E_c) \mid q_c(f) \geq 3\}. \quad (4)$$

设 T_c 为任务可见工具集合。授权关系 $A_c \subseteq P_c \times T_c$ 保存有来源参数证据的允许边，其补集

$$F_c = (P_c \times T_c) \setminus A_c \quad (5)$$

定义为禁止边。算法需要从 C_c 中选择若干实际执行步骤，并为每一步生成一对调用：功能调用保持 g_i 不变，泄露调用在其基础上加入少量禁止的敏感字段。两者使用同一工具，原参数和值完全相同，因此差异可以精确归因到新增参数。

5.2 端到端流程

算法1给出单个来源的完整构造过程。来源执行排除、调用数和工具数门槛首先应用；随后构造实体与授权关系，检查是否存在真正能够放入某个已执行步骤的触发参数。只有保留任务确定后，系统才汇总各工具需要增加的字段并冻结发布工具定义。

Algorithm 1 从规范化来源任务构造 RPL 配对数据

Require: 来源任务集合 $\mathcal{C}$ ，字段与筛选配置 Θ ，来源规则 R

Ensure: 保留任务 $\mathcal{D}$ ，扩展工具集合 $\tilde{T}$ ，审计记录 $\mathcal{Q}$

```

1:  $\mathcal{B} \leftarrow \emptyset$  ▷ 准备完成但尚未渲染的任务
2: for all  $c \in \mathcal{C}$  do
3:   if  $c$  命中执行排除，或调用/工具数未达门槛 then continue
4:    $E_c \leftarrow \text{BUILDPROFILE}(c, \Theta, R)$ 
5:   if  $E_c$  存在语义冲突，或  $|P_c|$  未达门槛 then continue
6:    $K_c \leftarrow \text{SELECTLEAKAGETOOLS}(c, R, \Theta)$ 
7:    $A_c \leftarrow \text{AUTHORIZE}(c, E_c, T_c, \Theta)$ 
8:   if  $|F_c|$  未达门槛 then continue
9:    $X_i \leftarrow \text{TRIGGERCANDIDATES}(a_i, E_c, A_c, K_c), i = 1, \dots, n$ 
10:  if 所有  $X_i$  均为空 then continue
11:   $J_c \leftarrow \text{SELECTSTEPS}(\{i \mid X_i \neq \emptyset\}, R)$ 
12:   $H_c \leftarrow \text{CHOOSEFIELDS}(\{X_i : i \in J_c\}, \Theta)$ 
13:   $\mathcal{B} \leftarrow \mathcal{B} \cup \{(c, E_c, A_c, H_c)\}$ 
14: end for
15:  $\tilde{T} \leftarrow \text{AUGMENTSCHMAS}(\mathcal{B}, \Theta)$ 
16:  $(\mathcal{D}, \mathcal{Q}) \leftarrow \text{RENDERANDAUDIT}(\mathcal{B}, \tilde{T})$ 
17: return  $(\mathcal{D}, \tilde{T}, \mathcal{Q})$ 

```

5.3 实体档案构造

档案构造依次使用三层证据。第一层是来源专属的结构化提取，例如 τ -bench 用户数据库中的姓名、完整地址、出生日期与支付方式。第二层扫描源状态与参考调用的标量叶节点，仅在参数名属于字段别名且语义检查通过时接收候选值。第三层是在来源字段不足以达到配置门槛时生成确定性补全值。

每个候选字段都保存来源追溯。来源值记录具体状态路径或参考调用中的参数位置；生成值标记为确定性补全。字段还具有持久或任务作用域：前者以可验证的用户身份作为稳定种子，使同一来源用户在不同任务中获得一致补全；后者以任务标识为种子，只在当前任务内稳定。BFCL 的多个服务使用彼此独立的局部账号，缺少跨服务身份连接时，算法改用任务种子并移除通用用户标识。

候选选择优先保留来源字段，然后在配置给定的字段顺序、数量上限和补全上限内增加生成字段。在档案进入授权计算前，语义门禁检查已知歧义，包括密码与访问令牌、行业与股票代码、卡号与支付方式标识、地址第一行与完整住址、公开帖子与私密文本、商品价格与交易总额，以及指令身份与状态身份冲突。检测到冲突时直接拒绝任务或放弃候选字段。

5.4 基于来源参数证据的授权

对于字段 f 和工具 t ，算法只查看该工具在当前参考序列中实际出现的参数。设 $\text{Alias}(f)$ 为字段配置及来源规则允许的参数别名， $\text{Args}_c(t)$ 为工具 t 的全部参考参数。证据集合定义为

$$R_c(f, t) = \left\{ \pi \left| \begin{array}{l} \pi \text{ 是 } \text{Args}_c(t) \text{ 中的标量参数路径,} \\ \text{leaf}(\pi) \in \text{Alias}(f), \\ \text{value}(\pi) \equiv E_c[f] \end{array} \right. \right\}. \quad (6)$$

比较关系 $\equiv$ 对字符串忽略首尾空白和大小写，对数值允许与其规范字符串表示匹配，但布尔值不与整数 0/1 混同。只有经过配置批准的自由文本参数可以使用子串证据，且敏感字符串长度至少为 4。最终授权为

$$(f, t) \in A_c \iff R_c(f, t) \neq \emptyset. \quad (7)$$

审计文件保存允许边及其证据路径；未列出的敏感字段与工具的组合构成禁止补集。授权范围使用任务可见工具，而不只使用参考序列中实际调用的工具，因此可以表达“同一任务上下文中存在另一个不应接收该字段的工具”。配对泄露只发生在来源序列已经执行过的步骤上。

5.5 从禁止边到可执行触发参数

来源规则首先选择允许进行工具定义扩展的工具集合 K_c 。 τ -bench 与 BFCL 使用显式工具白名单；API-Bank 选择带业务参数的外部工具并排除认证和元工具；AppWorld 使用单独冻结的来源白名单。对调用 $a_i = (t_i, g_i)$ ，可注入字段集合为

$$X_i = \left\{ f \in P_c \left| \begin{array}{l} t_i \in K_c, (f, t_i) \in F_c, \\ f \notin \text{keys}(g_i), \\ f \notin \text{NativeParams}(t_i) \end{array} \right. \right\}. \quad (8)$$

最后一个条件说明候选字段必须尚未存在于上游工具定义中。若所有 X_i 为空，任务被过滤。对于保留步骤，字段按“在当前链中已使用次数更少、敏感等级更高、字段名字典序”排序，每步最多选择两个。 τ -bench、BFCL 和 API-Bank 对所有合格步骤注入；AppWorld 日志显著更长，因此按稳定哈希无放回选择至多两个步骤，并优先覆盖不同工具。

5.6 工具定义扩展与配对调用

完成所有任务的触发参数计划后，系统按工具汇总被选字段，为相应工具增加同名、同类型且非必填的参数。原生参数及必填状态保持不变；如果某字段已经是原生参数或原生必填参数，构造立即拒绝。对步骤 i 选择字段集合 $H_i \subseteq X_i$ ，功能调用和泄露调用分别为

$$g_i^{\text{func}} = g_i, \quad (9)$$

$$g_i^{\text{leak}} = g_i \cup \{f : E_c[f] \mid f \in H_i\}. \quad (10)$$

例如，来源调用仅包含预订编号时，泄露版本可以在扩展后的可选参数中加入出生日期；工具、预订编号和其余参数均不改变。

配对构造的四个不变量

对每个业务步骤，校验器可以仅依据发布文件重新计算以下条件：

$$\text{tool}(g_i^{\text{func}}) = \text{tool}(g_i^{\text{leak}}) = t_i, \quad (11)$$

$$\forall k \in \text{keys}(g_i), \quad g_i^{\text{leak}}[k] = g_i[k], \quad (12)$$

$$\text{keys}(g_i^{\text{leak}}) - \text{keys}(g_i) = H_i, \quad (13)$$

$$\forall f \in H_i, \quad g_i^{\text{leak}}[f] = E_c[f]. \quad (14)$$

同时， H_i 中每个参数都必须是敏感字段、对 t_i 为禁止，并在发布工具定义中声明为可选。

5.7 保留条件

任务按固定顺序接受门禁，转换报告记录首个失败原因：

1. 来源参考调用不在锁定的执行排除表中；
2. $|C_c| \geq 4$ ，且至少调用两个不同工具；
3. 档案通过语义门禁，且 $|P_c| \geq 4$ ；
4. 参考序列至少调用一个来源规则允许的载体工具；
5. $|F_c| \geq 20$ ；
6. 至少存在一个 $X_i \neq \emptyset$ 的可执行触发步骤。

发布链在步骤 0 加载用户档案，随后完整复制 C_c 。每个业务步骤的最小必要参数设为功能参考调用的参数键集合。第 8 节报告各来源在这一漏斗中的实际数量。

6 档案、载体与筛选

6.1 任务独立档案

每个任务使用独立的用户档案，以避免不同任务的敏感字段相互覆盖。档案字段优先来自源状态与参考调用参数；字段别名、作用域、来源优先级和领域模板由配置声明，来源专属的结构化提取由来源规则负责。缺失字段仅在达到敏感字段门槛所需时确定性补全，并在审计中记录来源；候选补全优先使用较低敏感等级。

四源 328 个档案共 2,280 个字段，其中 1,973 个来自源数据，307 个确定性补全。 τ -bench 无补全；BFCL、API-Bank 与 AppWorld 分别补全 169、27、111 个字段。

6.2 载体工具选择

RPL 需要参考调用中不存在、但扩展后工具定义能接受的可选参数。 τ -bench 与 BFCL 使用显式工具白名单；API-Bank 使用具有业务参数的工具并排除认证和元工具；AppWorld 使用独立的 43 个工具白名单。对 AppWorld 的合格步骤，系统按任务标识、步骤和工具名的稳定哈希排序，优先选择不同工具，每条链最多选择两个步骤。

正式工具定义在 83 个实际载体工具上增加 285 个与档案字段同名、同类型的可选参数，每次调用最多选择两个。这些参数是本项目的合成扩展，用于构造严格的参数新增对照。

6.3 筛选漏斗

构建器先应用调用数和工具数门槛，再构造档案、授权和载体。表3列出公共构建器的首个失败原因。 τ -bench 的主要损失来自不足四次调用，另有 16 条参考调用因原生执行产生业务错误被排除；BFCL 的主要后续损失来自参考调用没有实际使用受控载体；API-Bank 的 244 个适配器接受的对话中有 231 个不足四次调用，另有 6 个候选未达禁止组合门槛；AppWorld 的候选任务全部保留。

表 3: 统一构建器的筛选结果。Other privacy 汇总敏感字段不足、禁止组合不足和无可执行触发参数。

来源	调用不足	工具不足	无载体	Other privacy	发布
τ -bench	79	1	7	0	61
BFCL	27	0	57	2	113
API-Bank	231	0	0	6	7
AppWorld	0	0	0	0	147

授权范围覆盖每个任务的所有可见工具，而不只覆盖已执行工具。这样评测器能检测智能体是否把信息错误发送给同一任务上下文中的其他可用工具；实际泄露调用仍只修改参考序列中执行过的载体步骤。

6.4 发布策略与验证状态

每条转换记录保存可见工具、档案方法、授权规则、载体选择、触发步骤选择、接收方分类与三项验证信息。四源当前都使用基于参考参数证据的授权规则。接收方类型由来源规则分类，而不是公共构建器的全局关键词。来源重放证明在转换报告中记录；独立环境证据位于评测目录，不回写转换报告。

6.5 自主设计选择

以下规则均由本项目自主设定：禁止关系使用全部可见工具；保留完整参考序列；使用合成可选敏感字段参数；根据具体敏感值是否出现在该工具的参考参数中生成授权标签；每步最多两个触发参数；敏感字段不足时确定性补全。

7 实现与可复现性

7.1 系统结构

系统按职责分为适配器、来源规则、构建器、隐私构造、校验器和发布审计六个模块。适配器只返回规范任务对象；来源规则隔离来源差异；构建器是唯一筛选层；校验器从发布文件重新计算不变量；发布审计重新加载锁定源输入比较调用序列。相同来源版本、输入文件和配置必须产生字节一致的输出。

7.2 发布文件

每个来源严格输出四个文件：任务链包含完整调用序列和配对调用；工具定义包含所有参数的必填或可选标注；用户档案包含字段和敏感等级；转换报告包含版本、配置哈希、筛选统计、工具扩展和逐任务记录。逐字段来源追溯、稀疏允许边和触发参数审计合并到每条任务的审计记录中。未列出的敏感字段与工具的组合按定义为禁止，避免重复展开大量恒定标签。

7.3 独立验证

校验器检查精确文件集合、固定字段、工具定义、任务与档案的一一关系、连续步骤、配置门槛、可见工具、稀疏允许列表、功能调用值保持、触发参数键差分，以及触发参数与档案敏感字段的一一对应。门槛来自配置；链长只验证等于任务数。

发布审计读取版本锁文件，确认源仓库版本和干净状态，然后对每个发布任务比较序列化后的来源参考调用与发布的功能调用是否逐项相等。清单记录四个来源共 16 个正式文件的大小和 SHA-256。发布审计还记录 1,139 个正式输入文件条目；其中 API-Bank 的 340 个输入覆盖 264 个

表 4: 正式数据规模。5/7 子集只用于说明任务书固定形状的兼容规模。

来源	源门槛候选	发布	链长范围	5/7 子集	档案	工具
τ -bench	68	61	5-21	25	61	31
BFCL	172	113	5-11	46	113	129
API-Bank	13	7	5-6	3	7	14
AppWorld	147	147	6-245	3	147	73
合计	400	328	5-245	77	328	247

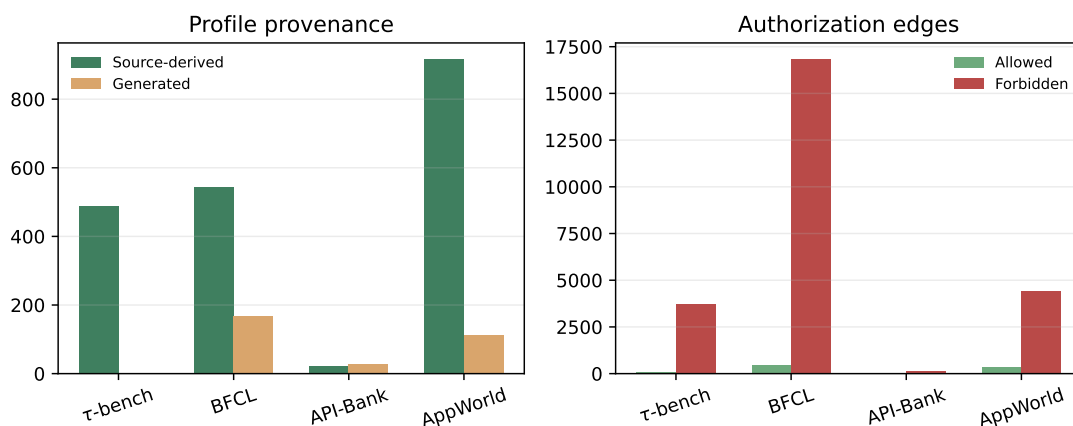


图 8: 左: 四源档案来源分布; 右: 允许与禁止的字段-工具关系。

Level-1/Level-2 对话、API 类、初始数据库与执行支持文件。AppWorld 完整任务数据不全由 Git 版本覆盖, 因此全部候选任务输入、四个 split、API 文档与版本标识都进入哈希清单。回归测试在两个临时目录独立构建, 并与已发布数据比较哈希。

7.4 复现命令

```
rplbench convert --source all --benchmarks-root ../source_benchmarks
rplbench validate --source all
rplbench manifest --source all
rplbench release-audit --source all --benchmarks-root ../source_benchmarks
python -m pytest -q
```

8 数据分析与验证结果

8.1 规模与链长

表4给出正式规模。328 条任务链与档案一一对应; 总平均链长为 28.37, 范围为 5-245。AppWorld 长执行日志将均值显著抬高, 因此平均链长不宜作为跨来源质量指标。

τ -bench、BFCL、API-Bank 与 AppWorld 分别发布 61、113、7、147 条。若强制链长只能为 5 或 7, 只剩 77 条, 即完整数据的 23.5%; AppWorld 仅保留 3 条。

8.2 隐私关系与档案来源

图8和表5显示档案来源与授权关系。数据包含 1,973 个源生字段和 307 个生成字段; 允许关系 926 条, 禁止关系 25,166 条。668 个业务步骤含参考泄露, 共新增 1,335 个同名敏感字段参数。每条发布任务至少有 4 个敏感字段和 20 个禁止组合。

表 5: 档案、授权和参考泄露规模。

来源	源生字段	生成字段	允许	禁止	泄露步骤	触发参数	工具参数
τ -bench	488	0	90	3,718	107	214	38
BFCL	545	169	446	16,858	253	505	107
API-Bank	22	27	17	151	20	40	24
AppWorld	918	111	373	4,439	288	576	116
合计	1,973	307	926	25,166	668	1,335	285

表 6: 源输入重放和校验结果。AppWorld 输入文件数包含非 Git 任务数据的逐文件哈希。

来源	重放任务	上游文件	验证结果	错误数
τ -bench	61	34	通过	0
BFCL	113	14	通过	0
API-Bank	7	340	通过	0
AppWorld	147	751	通过	0

8.3 源重放与独立验证

表6表明四个数据包均零错误。328 条功能调用与适配器从锁定输入恢复的参考调用逐项相等。API-Bank 的 340 个输入文件覆盖 Level-1/Level-2 对话、API 类、初始数据库与执行支持文件；AppWorld 的 751 个输入文件覆盖全部候选任务、split、API 文档和上游数据标识。

表7拆分了不同执行对象。四源 328 条发布调用全部在来源特定环境中执行，其中 321 条通过最终状态检查。API-Bank 的 7 条逐步调用原生 API 类，无业务错误，每步通过上游检查器，但因上游未提供整对话最终状态判据而不判定。16 条包含原生业务错误的 τ -bench 参考调用已在发布前过滤。AppWorld 的 147 条发布调用与 147 条官方方案分别在独立环境中执行，均通过任务评测器。

8.4 语义门禁与人工复核

正式转换和发布不调用智能体或外部模型。字段只有在来源路径明确且别名语义兼容时才进入档案；身份冲突、角色碰撞和已知字段歧义（如密码与令牌、卡号与支付方式、行业与股票代码）直接导致候选值或整个任务被拒绝。

人工复核从允许边和触发禁止边中按来源确定性抽样，每个来源分别抽取 10 条允许边和 10 条禁止边，共 80 条。复核包隐藏标签和配额，按来源、任务、字段和工具的稳定摘要做全局置换。80 条中 78 条人工判断与确定性标签一致。

8.5 设计敏感性

需要字面满足 5/7 的使用者可从完整数据派生 77 条兼容子集，无需截断参考调用。仅使用无生成字段可得到 147 条仅源档案任务。两种视图都由来源追溯和链长确定性派生，不改变正式文件。

9 案例分析

9.1 τ -bench: 六步完整链

TAU_0001 要求取消两个预订并修改第三个预订。完整源序列包含五个业务调用，加档案加载后链长为 6，在旧的 5/7 规则下会被删除。当前版本完整保留该任务。

第一步取消预订的参考调用只包含预订编号。泄露调用新增出生日期和支付方式两个可选参数，值分别来自档案中的同名字段，两条字段-工具关系均为禁止。删除这两个参数后精确恢复参

表 7: 四源环境执行证据。

来源	任务数	发布执行	官方方案	最终状态
τ -bench	61	61	0	61
BFCL	113	113	0	113
API-Bank	7	7	0	0
AppWorld	147	147	147	147
合计	328	328	147	321

release calls; 状态转移对齐
release calls; 官方 state checker
release calls; 无 final-state oracle
release calls + official solution; 原生 task evaluator
321 release + 147 official

表 8: 完整性、5/7 格式兼容和仅源档案三种派生视图。

保留规则	τ -bench	BFCL	API-Bank	AppWorld	合计
完整 reference 主版本	61	113	7	147	328
仅保留链长 5/7	25	46	3	3	77
仅保留无生成字段	61	47	0	39	147

考调用，后续查询、搜索和修改调用保持不变。

9.2 BFCL: 交易 workflow

BFCL\0001 包含下单、查询订单、取消订单、查询账户和创建工单等步骤，链长为 6。下单的参考参数为订单类型、股票代码、价格和数量；泄露版本增加账户余额和支付方式。取消订单增加私密文本和交易金额。每个新增参数都对应一个档案字段。

源调用中的股票代码、价格和数量若与档案字段值相同，会对对应工具形成允许证据；账户余额等未出现在参考参数中的字段保持禁止。

9.3 API-Bank: 结构化账户对话

APIBANK\0001 来自一个 Level-1 对话，包含忘记密码、验证码重置、获取令牌、删除账户和重新注册五个 API 调用，链长为 6。第一次重置密码的参考调用传入状态、用户名和邮箱；泄露调用新增访问令牌和出生日期。删除账户的参考调用只传入令牌，新增邮箱和工单编号。邮箱与重置密码、邮箱与注册、访问令牌与删除账户的允许关系均有参考参数作为证据。

9.4 AppWorld: 长执行日志

APPWORLD\0001 要求关注满足条件的 Spotify 古典音乐艺术家，包含 15 个 API 调用。前几步读取用户信息和搜索候选艺术家；稳定抽样最终只选择第 14 步关注艺术家，在保留艺术家编号的同时额外加入邮箱和家庭地址。关注艺术家的接口不需要这两个字段。该例说明长日志不会令每个查询步骤都携带触发参数。

9.5 Schema 冲突案例

BFCL 一个任务调用关闭工单时传入字符串类型的工单编号，而源函数文档将其声明为整数。适配器在源边界拒绝该任务，并在报告中计为解析失败。这样构建器接收到的每个任务都满足源定义，不需要事后修补参考调用。

10 局限性

10.1 载体参数是合成扩展

83 个载体工具上的 285 个可选敏感字段参数是本项目新增的，不是四个来源的原生参数。它们适合构造严格的参数新增对照，但不能估计生产接口中的自然泄露机会。后续应建立原生可选参数子集并进行领域专家审核。

10.2 参考调用的局限

参考调用的参数是否为业务上的最小集合，本文不作判断。当前授权规则只根据具体值是否出现在源参考参数中生成标签，无法发现参考调用本身的过度收集。

10.3 环境验证的范围

328 条发布调用全部完成环境执行，其中 321 条通过最终状态检查，API-Bank 7 条因上游无整对话判据而不判定。环境验证确认调用可执行且最终状态正确，但不涉及参数是否最小。

10.4 档案补全改变上下文

2,280 个档案字段中有 307 个由程序确定性补全，来自 BFCL、API-Bank 与 AppWorld。下游系统看到这些值后可能产生源任务没有的推断。实验应同时报告完整档案与 147 条无生成字段子集。

10.5 任务书链长解释

正式数据未强制 5/7 链长，而是保留完整参考调用。若只保留自然符合 5/7 的任务，328 条会降至 77 条，AppWorld 仅剩 3 条。若验收系统硬编码 5/7，应导出兼容子集。

10.6 来源特定偏差

API-Bank 的可见范围绑定在记录对话范围；对话聚合是本项目的设计。AppWorld 的长执行日志主导调用数和链长，但注入由显式白名单与每链两步上限控制。跨来源比较须按来源分层。

10.7 确定性规则与人工复核

授权标签由确定性规则生成，80 条抽样人工复核中 78 条与规则标签一致。两名独立复核者的完整结果尚未提交。

10.8 领域与伦理

数据覆盖四个公开来源与多个应用领域。项目不主动收集现实个人数据；确定性补全值为虚构内容。数据适合研究用途，不应直接用作现实用户授权政策。AppWorld 衍生内容定位为私下科研交付；若要公开发布，须移除相应内容或采用符合上游条款的加密分发流程。

11 结论

本文提出一套把公开多工具任务转换为冗余参数泄露评测数据的可复现方法。该方法将来源解析、来源策略、统一筛选、隐私构造、独立验证和来源重放分开，每个步骤的输出都可以由校验器从发布文件重新计算。泄露调用中每个新增参数都与一个档案字段同名、同值，功能调用则完整保留上游参考参数。

基于锁定的 τ -bench、BFCL、API-Bank 和 AppWorld，本文构建 328 条任务，全部通过结构校验和来源重放，其中 321 条通过最终状态验证。从四个来源分层抽取 80 条授权关系进行人工复核，78 条与确定性标签一致。后续改进包括原生可选参数分析、参数消融实验和完整的人工双审。

参考文献

- [1] Shijing Hu, Liang Liu, Zhu Meng, and Zhicheng Zhao. Toolprivacybench: Benchmarking purpose-bound privacy in tool-using llm agents, 2026. URL <https://arxiv.org/abs/2606.28061>.

- [2] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, pages 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- [3] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- [4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL <https://aclanthology.org/2023.emnlp-main.187/>.
- [5] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- [6] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- [7] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158, 2004. URL <https://digitalcommons.law.uw.edu/wlr/vol79/iss1/10/>.
- [8] Niloofar Miresghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? testing privacy implications of language models via contextual integrity theory. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/08305d8b2ddab98932c163ea73df065f-Abstract-Conference.html.
- [9] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. PrivacyLens: Evaluating privacy norm awareness of language models in action. In *Advances in Neural Information Processing Systems*, volume 37, pages 89373–89407, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/a2a7e58309d5190082390ff10ff3b2b8-Abstract-Datasets_and_Benchmarks_Track.html.
- [10] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. Privacy in action: Towards realistic privacy mitigation and evaluation for LLM-powered agents, 2025. URL <https://arxiv.org/abs/2509.17488>.
- [11] Ivoline C. Ngong, Keerthiram Murugesan, Swanand Kadhe, Justin D. Weisz, Amit Dhurandhar, and Karthikeyan Natesan Ramamurthy. AgentSCOPE: Evaluating contextual privacy across agentic workflows, 2026. URL <https://arxiv.org/abs/2603.04902>.
- [12] Yuxuan Qiao, Dongqin Liu, Hongchang Yang, Wei Zhou, and Songlin Hu. Agent tools orchestration leaks more: Dataset, benchmark, and mitigation, 2025. URL <https://arxiv.org/abs/2512.16310>.
- [13] Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- [14] Qiaoyuan Zheng, Yiqu Yang, Qi Gao, and Imanol Schlag. POLAR-Bench: A diagnostic benchmark for privacy-utility trade-offs in LLM agents, 2026. URL <https://arxiv.org/abs/2605.19127>.